

14-15 mavzular. Assembler tilida dasturlash asoslari

Kirish

Assembler tili — bu **quyi darajadagi dasturlash tili** bo'lib, u bevosita protsessor buyruqlariga yaqin tarzda ishlaydi. Assembler tili yordamida yozilgan dasturlar kompyuter apparatiga juda yaqin bo'lib, yuqori darajadagi tillarga nisbatan tezroq ishlaydi va apparat resurslarini aniq boshqarish imkonini beradi. Shu sababli assembler tili operatsion tizimlar yadrosi, qurilma drayverlari, o'rnatilgan tizimlar (embedded systems) va real vaqt tizimlarida keng qo'llaniladi.

Assembler tilini o'rganish talabalarga protsessor qanday ishlashini, buyruqlar qanday bajarilishini va xotira bilan qanday ishlash mexanizmlarini chuqur tushunish imkonini beradi. Ushbu mavzuda assembler tilining imkoniyatlari, operatorlari va assemblerlash jarayoni asosiy tushunchalar bilan birga yoritiladi.

1. Assembler dasturlash tili tushunchasi

Assembler tili — bu mashina tilining **mnemonik ko'rinishi** bo'lib, protsessor buyruqlari odam uchun tushunarli qisqartmalar yordamida ifodalanadi. Masalan, ADD, MOV, SUB, JMP kabi buyruqlar mashina kodidagi murakkab ikkilik ketma-ketliklarning mnemonik ifodasidir.

Assembler tili **apparatga bog'liq** bo'lib, har bir protsessor arxitekturasida (x86, ARM, AVR va boshqalar) o'z assembler buyruqlar to'plamiga ega. Shu sababli bir protsessor uchun yozilgan assembler dasturi boshqa arxitektura to'g'ridan-to'g'ri ishlamaydi.

Assembler tilining asosiy xususiyatlari:

- protsessor buyruqlariga to'g'ridan-to'g'ri moslik
- yuqori tezlik va samaradorlik
- xotira va registrlarni aniq boshqarish
- apparatga yaqin dasturlash imkoniyati

2. Assembler tilining imkoniyatlari

Assembler tili quyidagi imkoniyatlarni taqdim etadi:

- protsessor registrlari bilan bevosita ishlash
- kiritish-chiqarish portlarini boshqarish
- uzilishlar (interrupt) bilan ishlash

- xotirani past darajada boshqarish
- yuqori darajadagi tillarda imkoni bo'lmagan optimallashtirishlarni amalga oshirish

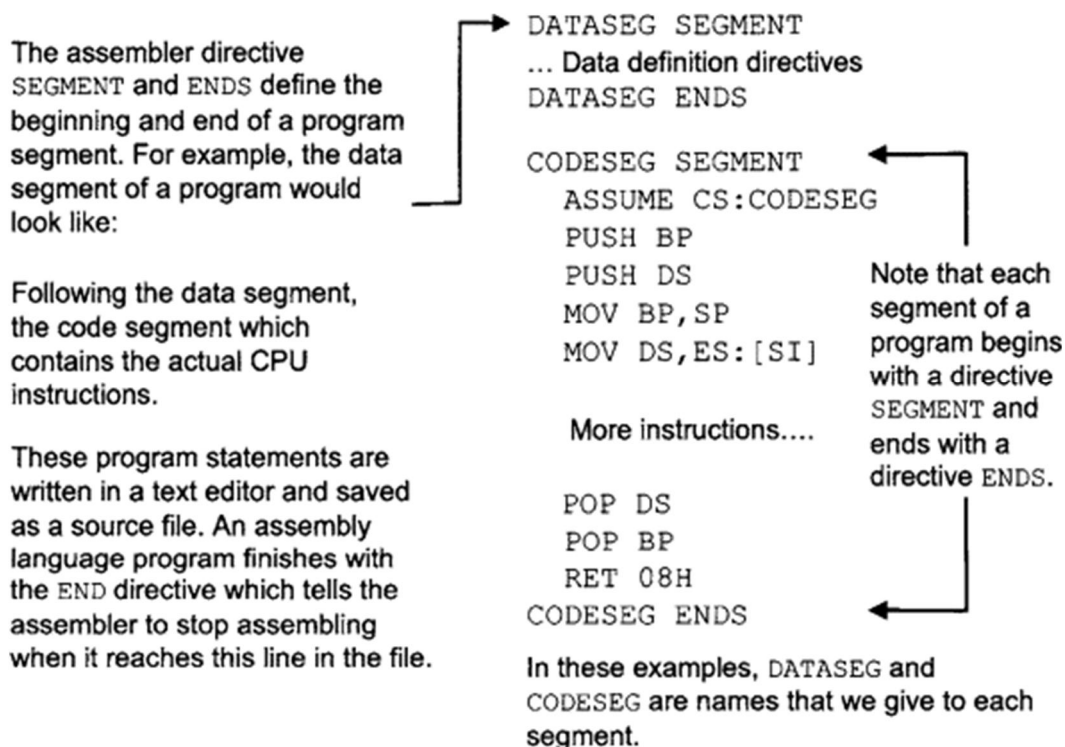
Assembler tilida yozilgan dasturlar kichik hajmli va tezkor bo'ladi. Shu bilan birga, assembler dasturlash murakkab bo'lib, dasturchidan apparat tuzilishini yaxshi bilishni talab qiladi.

3. Assembler dasturining umumiy tuzilishi

Assembler dasturi odatda quyidagi qismlardan iborat bo'ladi:

- **ma'lumotlar bo'limi** (data segment)
- **kod bo'limi** (code segment)
- **stek bo'limi** (stack segment)

Ma'lumotlar bo'limida o'zgaruvchilar va konstantalar e'lon qilinadi. Kod bo'limida buyruqlar ketma-ketligi yoziladi. Stek bo'limi esa vaqtinchalik ma'lumotlar va chaqiruvlar uchun ishlatiladi.



Assembler tilida dasturlashni chuqurroq o'rganishda eng muhim jihatlardan biri — bu **protssessor arxitekturasini bilan uzviy bog'liqlik**dir. Assembler dasturchi nafaqat dastur mantiqini, balki protssessorning ichki ishlash mexanizmlarini ham aniq

tushunishi talab etiladi. Masalan, buyruqlar qanday ketma-ketlikda bajarilishi, qaysi registrlar qaysi vazifani bajarishi va qaysi buyruqlar flaglar holatiga ta'sir qilishi muhim ahamiyatga ega.

Assembler dasturlashda **registrlardan samarali foydalanish** alohida e'tiborga loyiq. Registrlar soni cheklangan bo'lgani sababli ularni noto'g'ri boshqarish dastur samaradorligini pasaytiradi. Tajribali assembler dasturchilar imkon qadar xotiraga murojaat qilishni kamaytirib, barcha vaqtinchalik hisoblashlarni registrlar orqali amalga oshiradi. Bu esa dastur bajarilish tezligini sezilarli darajada oshiradi.

Assembler dasturlarda **xotira bilan ishlash** ham muhim masalalardan biridir. Xotira adreslari bilan bevosita ishlash, ko'rsatkichlar (pointer) tushunchasi va massivlar bilan ishlash assemblerda ancha murakkab, ammo juda kuchli imkoniyatlarni taqdim etadi. Dasturchi xotiraning qaysi qismi qayerda joylashganini aniq bilishi va unga mos ravishda adreslash usullaridan foydalanishi lozim. Bevosita, bilvosita va indeksli adreslash usullari assembler dasturlarining moslashuvchanligini oshiradi.

Assembler tilida **shartli va shartsiz o'tishlar** dastur mantiqining asosini tashkil etadi. Yuqori darajadagi tillarda mavjud bo'lgan if, while, for kabi konstruktsiyalar assemblerda flaglar va shartli sakrash buyruqlari yordamida amalga oshiriladi. Bu jarayon dasturchidan mantiqiy fikrlashni va dastur oqimini oldindan rejalashtirishni talab qiladi. Noto'g'ri tashkil etilgan sakrashlar dasturda cheksiz sikllar yoki noto'g'ri ishlash holatlariga olib kelishi mumkin.

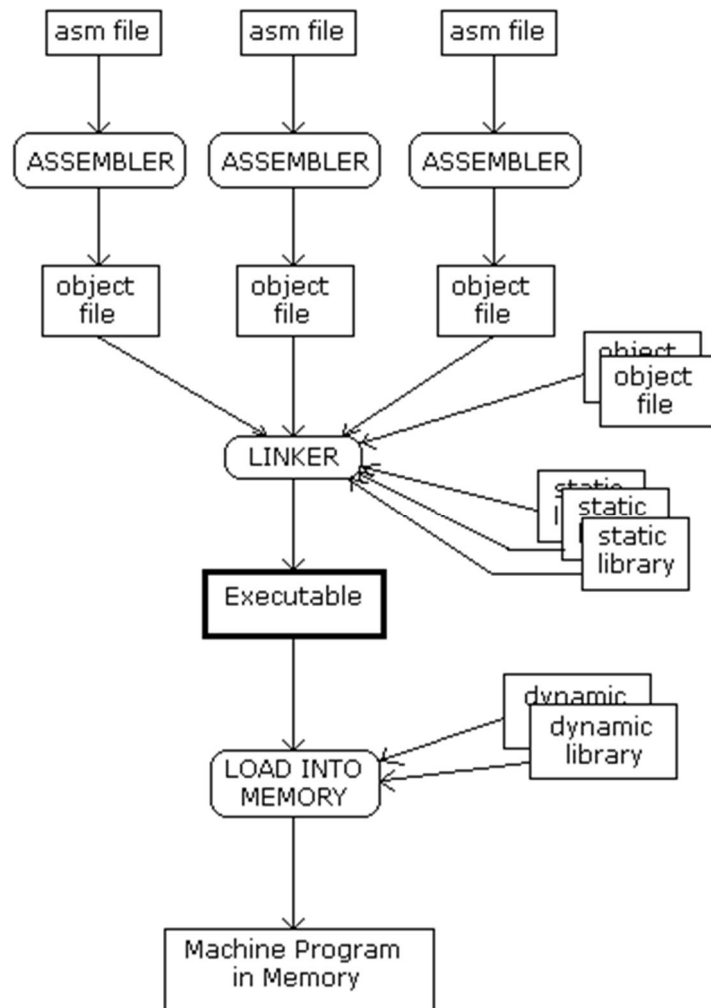
Assembler dasturlashda **stek bilan ishlash** alohida ahamiyatga ega. Stek yordamida funksiyalarni chaqirish, parametrlarni uzatish va lokal o'zgaruvchilarni saqlash amalga oshiriladi. PUSH va POP buyruqlari orqali stekdan foydalanish dastur bajarilishining izchilligini ta'minlaydi. Ayniqsa, uzilishlar bilan ishlashda stek mexanizmi muhim rol o'ynaydi, chunki uzilish sodir bo'lganda protsessor joriy holatni stekda saqlab qo'yadi.

Assembler tilida **uzilishlar bilan ishlash** apparat darajasidagi dasturlashning muhim qismi hisoblanadi. Uzilishlar tashqi qurilmalar bilan samarali ishlash imkonini beradi. Assembler dasturchi uzilish vektorlarini, xizmat dasturlarini (ISR) va uzilishlarni yoqish/o'chirish mexanizmlarini yaxshi bilishi kerak. Bu bilimlar real vaqt tizimlari va o'rnatilgan tizimlarda ayniqsa muhimdir.

Assembler dasturlarini yozishda **optimallashtirish** muhim masala hisoblanadi. Optimallashtirish nafaqat tezlikni oshirish, balki xotira sarfini kamaytirish va kod hajmini kichraytirish uchun ham amalga oshiriladi. Ba'zan bir xil vazifani bajaruvchi bir nechta buyruqlar ketma-ketligi mavjud bo'lishi mumkin, ammo ularning samaradorligi turlicha bo'ladi. Tajribali assembler dasturchi eng qisqa va eng tezkor variantni tanlay oladi.

Assembler tilida **makroslar** va **direktivlar** keng qo'llaniladi. Makroslar takrorlanuvchi buyruqlar ketma-ketligini soddalashtirishga yordam beradi.

Direktivlar esa assemblerga qanday ishlash kerakligini ko'rsatadi, masalan, xotira ajratish, segmentlarni aniqlash yoki konstantalarni belgilash uchun ishlatiladi. Bu vositalar assembler dasturlarining o'qiluvchanligini va boshqarilishini yaxshilaydi.



4. Assembler tili operatorlari (buyruqlari)

Assembler tilidagi operatorlar protsessor bajaradigan amallarni ifodalaydi. Ular bir nechta asosiy guruhlariga bo'linadi.

4.1. Ma'lumotlarni uzatish buyruqlari

Bu buyruqlar ma'lumotlarni registrlar va xotira o'rtasida ko'chirish uchun ishlatiladi. Eng mashhur buyruqlardan biri MOV bo'lib, u ma'lumotni bir joydan ikkinchi joyga ko'chiradi.

4.2. Arifmetik buyruqlar

Arifmetik buyruqlar qo'shish, ayirish, ko'paytirish va bo'lish amallarini bajaradi. Masalan, ADD, SUB, INC, DEC buyruqlari.

4.3. Mantiqiy buyruqlar

Mantiqiy buyruqlar bitlar ustida amallar bajaradi. AND, OR, XOR, NOT buyruqlari ushbu guruhga kiradi.

4.4. Taqqoslash va shartli o'tish buyruqlari

Bu buyruqlar dastur bajarilish ketma-ketligini boshqaradi. CMP buyruği taqqoslashni amalga oshiradi, JZ, JNZ, JC kabi buyruqlar esa shartli o'tishni ta'minlaydi.

4.5. Boshqaruv buyruqlari

JMP, CALL, RET kabi buyruqlar dastur oqimini boshqarish uchun ishlatiladi.

5. Registrlar bilan ishlash

Assembler tilida dasturlashda registrlar asosiy rol o'ynaydi. Registrlar juda tezkor bo'lib, hisoblash amallari asosan ular orqali amalga oshiriladi. Har bir registrning aniq vazifasi mavjud bo'lib, noto'g'ri ishlatilishi dastur xatolariga olib kelishi mumkin.

6. Assemblerlash jarayoni

Assemblerlash jarayoni — bu assembler tilida yozilgan dastur matnini mashina kodiga aylantirish jarayonidir. Ushbu jarayon maxsus dastur — **assembler** yordamida amalga oshiriladi.

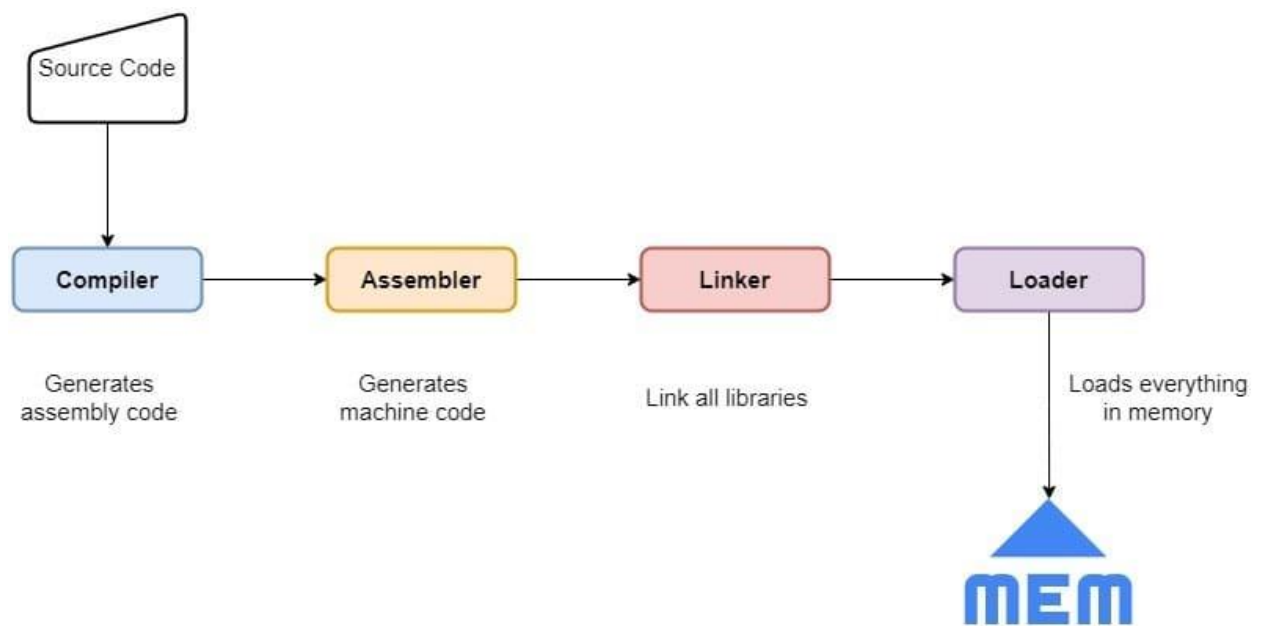
Assemblerlash jarayoni quyidagi bosqichlardan iborat:

1. manba kodni tahlil qilish
2. mnemonik buyruqlarni mashina kodiga aylantirish
3. simbolik nomlarni adreslarga almashtirish
4. obyekt fayl yaratish

Assembler dasturlashni o'rganishda **xatolar bilan ishlash** ham muhim bosqich hisoblanadi. Sintaksis xatolari, mantiqiy xatolar va apparatga bog'liq xatolar assemblerda tez-tez uchraydi. Shu sababli debuggerlardan foydalanish, registrlar va

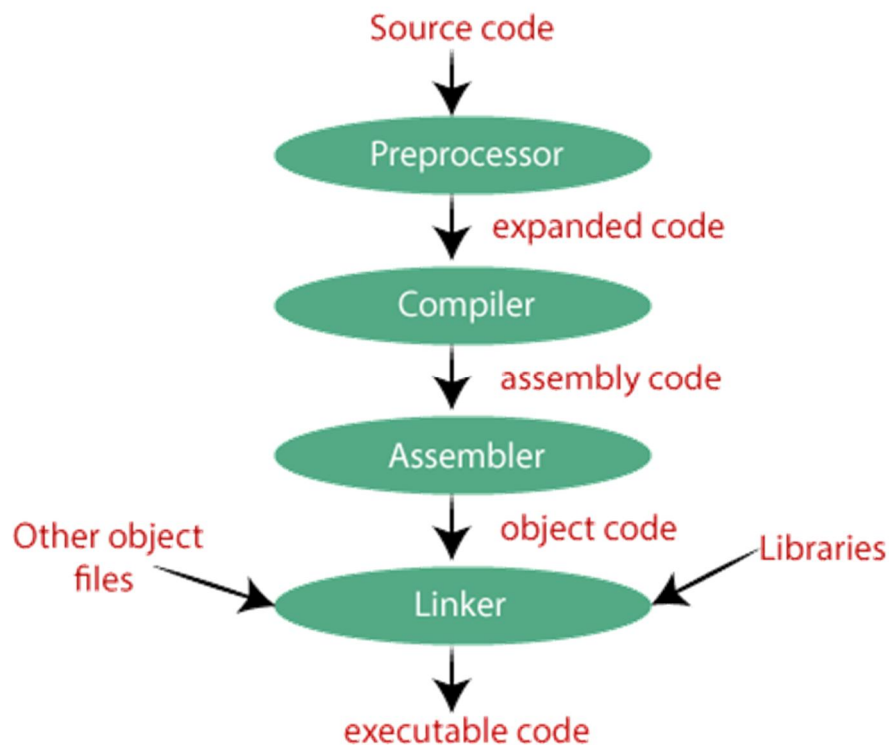
xotira holatini tekshirish assembler dasturchining asosiy ko'nikmalaridan biri bo'lishi kerak. Debug jarayoni dasturchiga dastur qanday ishlayotganini chuqur tushunish imkonini beradi.

Assembler tilining amaliy qo'llanilish sohalari juda keng. Bugungi kunda assembler operatsion tizimlar yadrosida, BIOS va UEFI dasturlarida, qurilma drayverlarida, mikroprotssessorli va mikrokontrollerli tizimlarda keng qo'llaniladi. Ayniqsa, resurslari cheklangan qurilmalarda assembler tilining ahamiyati juda katta, chunki u apparat resurslaridan maksimal darajada foydalanish imkonini beradi.



Yuqori darajadagi tillar rivojlanganiga qaramay, assembler tili o'z ahamiyatini yo'qotgani yo'q. Aksincha, assembler zamonaviy dasturlashda muhim qo'shimcha vosita sifatida qaraladi. Ko'plab kompilyatorlar assembler darajasida optimallashtirishlarni amalga oshiradi va dasturchilar ayrim kritik qismlarni qo'lda assemblerda yozishni afzal ko'radi.

Sodda qilib aytganda, assembler tilida dasturlash asoslarini chuqur o'zlashtirish dasturchiga kompyuter tizimining ichki ishlash mexanizmlarini to'liq tushunish imkonini beradi. Assembler bilimlari apparatga yaqin dasturlar yaratishda, tizimli dasturlashda va samaradorlik muhim bo'lgan sohalarda katta ustunlik beradi. Ushbu bilimlar keyingi bosqichlarda operatsion tizimlar, kompyuter arxitekturasini va o'rnatilgan tizimlarni o'rganishda mustahkam poydevor bo'lib xizmat qiladi.



Assemblerlashdan so‘ng linklash va yuklash (loader) jarayonlari amalga oshiriladi.

7. Assembler va yuqori darajadagi tillar taqqoslanishi

Assembler tili yuqori darajadagi tillarga nisbatan tezroq ishlaydi, ammo dasturlash jarayoni murakkabroq va ko‘proq vaqt talab qiladi. Yuqori darajadagi tillar esa qulaylik va ko‘chma kod yaratish imkonini beradi.

Assembler ko‘pincha yuqori darajadagi tillar bilan birgalikda qo‘llaniladi, ya’ni muhim va tezkor qismlar assemblerda, qolgan qismi esa C/C++ kabi tillarda yoziladi.

Xulosa

Assembler tilida dasturlash asoslari protsessorning ichki ishlash mexanizmlarini chuqur tushunishga yordam beradi. Assembler tili yuqori tezlik, aniq boshqaruv va apparatga yaqinlikni ta’minlaydi. Ushbu tilni o‘rganish kompyuter arxitekturasini, operatsion tizimlar va o‘rnatilgan tizimlar sohasida muhim bilimlar beradi.

14-mavzu bo'yicha 10 ta nazorat savollari

1. Assembler tili nima va u nima sababdan quyi darajadagi dasturlash tili hisoblanadi?
2. Assembler tilining asosiy imkoniyatlari va afzalliklari nimalardan iborat?
3. Nima sababdan assembler tili apparatga bog'liq hisoblanadi?
4. Assembler dasturining asosiy bo'limlarini sanab bering.
5. Ma'lumotlarni uzatish buyruqlari qanday vazifani bajaradi?
6. Arifmetik va mantiqiy buyruqlar o'rtasidagi farq nimada?
7. Shartli o'tish buyruqlari dastur bajarilishiga qanday ta'sir ko'rsatadi?
8. Registrlar assembler dasturlashda nima uchun muhim hisoblanadi?
9. Assemblerlash jarayoni nima va u qaysi bosqichlardan iborat?
10. Assembler va yuqori darajadagi dasturlash tillari o'rtasidagi farqlarni tushuntiring.